JURNAL RISET INFORMATIKA **P-ISSN: 2656-1743 |E-ISSN: 2656-1735**
Vol. 9, No. 1. Desember 2026 DOI: https://doi.org/10.34288/jri.v6i1.XXX
Accredited rank 4 (SINTA 4), excerpts from the decision of the DITJEN DIKTIRISTEK No. 230/E/KPT/2023

Time-Based Coffee Shop Menu Recommendation System Using K-Nearest Neighbor

Yusuf Herfa Auliansyah[1], Yusuf Mansur[2], Rizqi Nuzul Firman[3]

Informatics Study Program (S1)
Universitas Bina Sarana Informatika (UBSI), Kaliabang Campus, Bekasi, Indonesia
yusufherfaaa@gmail.com; 15240047@bsi.ac.id; 15240116@bsi.ac.id

Abstract

Coffee shops that show every customer the same long menu at every hour give little guidance, even though what people order and how busy the shop is change through the day. This study designs and implements a time-based menu recommendation system, with an hourly crowd prediction, for Lower Manhattan Coffee Shop using the K-Nearest Neighbor (KNN) algorithm. The system is a web application built on CodeIgniter 4 in which customers order from their table through a QR code, and a Python Flask service serves the scikit-learn models. The models were trained on the public Coffee Shop Sales dataset from Maven Analytics, a simulated dataset, using the 47,782 transactions of the Lower Manhattan branch from January to June 2023. After data quality tests, hour, day of the week, and a weekend indicator were used as features. KNN classification predicts the product category, and KNN regression predicts the number of transactions per hour. Both were compared with simple baselines. Category classification used K = 20 and reached 34.89% accuracy, below the 38.10% of the majority-class baseline. Crowd prediction used K = 17 and reached a mean absolute error of 6.12 transactions per hour and an R^2 of 0.619, better than the baseline values of 7.42 and 0.417. In this dataset, the time of day explains how busy the shop is much better than what customers choose. A rule-based fallback keeps recommendations available when the model service is unreachable. Validation with real transactions is the next step.

Keywords: recommendation system; K-Nearest Neighbor; time-based; coffee shop; crowd prediction

Abstrak

Coffee shop yang menampilkan daftar menu panjang yang sama pada setiap jam kurang membantu pelanggan, padahal apa yang dipesan dan tingkat keramaian berubah sepanjang hari. Penelitian ini merancang dan membangun sistem rekomendasi menu berbasis waktu beserta prediksi keramaian per jam untuk Lower Manhattan Coffee Shop menggunakan algoritma K-Nearest Neighbor (KNN). Sistem berupa aplikasi web berbasis CodeIgniter 4 tempat pelanggan memesan dari meja melalui kode QR, dengan layanan Flask berbasis Python yang menjalankan model scikit-learn. Model dilatih pada dataset publik Coffee Shop Sales dari Maven Analytics, yaitu dataset simulasi, menggunakan 47.782 transaksi cabang Lower Manhattan pada Januari sampai Juni 2023. Setelah pengujian kualitas data, fitur yang dipakai adalah jam, hari dalam seminggu, dan penanda akhir pekan. Klasifikasi KNN memprediksi kategori produk, sedangkan regresi KNN memprediksi jumlah transaksi per jam. Keduanya dibandingkan dengan baseline sederhana. Klasifikasi kategori memakai K = 20 dan mencapai akurasi 34,89%, di bawah baseline kelas mayoritas sebesar 38,10%. Prediksi keramaian memakai K = 17 dan mencapai galat absolut rata-rata 6,12 transaksi per jam dengan R^2 0,619, lebih baik daripada nilai baseline 7,42 dan 0,417. Pada dataset ini, waktu menjelaskan tingkat keramaian jauh lebih baik daripada pilihan menu pelanggan. Mekanisme cadangan berbasis aturan menjaga rekomendasi tetap tersedia saat layanan model tidak dapat dijangkau. Validasi dengan transaksi nyata menjadi langkah berikutnya.

Kata kunci: sistem rekomendasi; K-Nearest Neighbor; berbasis waktu; coffee shop; prediksi keramaian

INTRODUCTION

A coffee shop with a long menu gives customers many choices but little guidance, and the items that sell well at breakfast are rarely the ones that sell well near closing time. Lower Manhattan Coffee Shop, the case of this study, offers 80 menu items in nine categories, from coffee, tea, and bakery items to packaged beans. Customers scan a QR code at

The work is distributed under the Creative Commons Attribution-NonCommercial 4.0 International License

their table to order without creating an account, and the cashier handles payment and stock through the same application. Digital ordering is already reaching small coffee shops. Arisandi et al. (2024) replaced handwritten order records at a coffee shop with an ordering application, and Salam et al. (2026) describe a QR-based self-ordering system at a warkop, a small Indonesian coffee stall. Pratama and Dewi (2025) mined the sales records of a coffee shop with the Apriori algorithm to understand purchasing patterns. These works cover ordering, record keeping, and item associations, but not what the menu should highlight at a given hour. In the application, the menu is shown in the same order all day, so a customer who wants a suggestion has to scroll through the whole list.

Recommender systems are the usual answer to this problem. Meng et al. (2022) survey context-aware recommender systems and the way they are evaluated, and Bogina et al. (2023) survey how temporal aspects such as the time of day are used in recommendation. For food, Bondevik et al. (2024) reviewed 67 studies and found that food recommender systems vary widely, mostly rely on content-based filtering and machine learning, and often give non-personalized suggestions. Chow et al. (2023) reached a similar picture in their review of techniques, datasets, and evaluation metrics. Among time-aware work, Panwar et al. (2024) built a food recommender with a support vector machine that combines the time of day with a weekend multiplier, and they report better error values than a rule-based baseline. Their setting is different from a QR ordering application, however. It predicts ratings for individual users, while a table-ordering application usually has no customer accounts and its orders are anonymous. A non-personalized recommendation that depends on when the customer orders is therefore more realistic here.

The K-Nearest Neighbor (KNN) algorithm is a reasonable fit for this setting. It classifies an item by the labels of its closest examples, needs no complicated training step, and its decisions can be explained to a shop owner by pointing to similar past orders. Hssina et al. (2021) combined KNN with singular value decomposition to address the cold start problem in a collaborative filtering recommender. Indonesian studies have also applied KNN to food businesses. Sari et al. (2025) used KNN with k = 3 on one year of sales data from a Padang restaurant to classify how well each menu item sells and reported 95.45% accuracy. Nafi and Mujiyono (2025) used KNN to suggest daily meals from nutritional values and user preferences, and Mellany and Kusnawi (2026) combined popularity ranking, TF-IDF with KNN, and tag-based similarity to recommend menu items from about 77,000 transactions of a small restaurant. Rizki et al. (2024) used Apriori to recommend menu bundles from restaurant transactions. These studies rely on sales classification, nutrition, item similarity, or association rules. None of them takes explicit time-of-day segments as the basis of the recommendation.

KNN also has two sensitive choices that this study has to handle. The first is feature scaling. Allorerung et al. (2024) compared normalization methods for KNN on three disease datasets and found that the best method depended on the dataset. The second is the number of neighbors K. Hafid (2023) and Pratama and Mandita (2024) used k-fold cross-validation to examine KNN performance and to select K, and Taslim et al. (2022) reported a modest accuracy of about 56.67% with K = 5 on a small student dataset.

The work is distributed under the Creative Commons Attribution-NonCommercial 4.0 International License

We therefore standardize the features and select K by cross-validation instead of fixing it in advance.

This study also faces a data limit. Lower Manhattan Coffee Shop has no history of real transactions yet, so the models are trained on the public Coffee Shop Sales dataset from Maven Analytics, which describes a simulated coffee shop with three branches. Figueira and Vaz (2022) review methods for generating synthetic data and assessing its quality, and Hidasi and Czapp (2023) show that flawed offline evaluation setups are common in recommender systems research. Both are reminders that results on simulated data need careful reading and comparison with simple baselines. A further question follows. If what customers order depends only weakly on the hour, a time-based menu recommendation has little to learn, whereas the number of customers per hour may still follow a clear daily pattern. We therefore pair the menu recommendation with a crowd prediction, and we report both against baselines so that the contribution of KNN is not overstated.

The objectives of this study are (1) to design a time-based menu recommendation that uses KNN with four time segments, namely morning (06.00-10.59), afternoon (11.00-14.59), evening (15.00-17.59), and night (18.00-20.59); (2) to predict the number of transactions per hour with KNN regression and to label each hour as busy, moderate, or quiet; (3) to integrate both models into the ordering application through a Flask service, following the model deployment approach of Suherlan et al. (2025), and a CodeIgniter 4 API, with a rule-based fallback when the service is unavailable; and (4) to evaluate the models with accuracy, macro F1-score, mean absolute error, and R^2 against simple baselines. The expected benefit is a shorter path from opening the menu to ordering for customers, and a clearer view of time-dependent demand for the owner.

RESEARCH METHODS

Types of Research

This research is applied quantitative research with two parts. The first part is system development: a time-based recommendation service is built and connected to a table-ordering web application. The second part is an offline experiment: two K-Nearest Neighbor (KNN) models are trained and evaluated on secondary transaction data and compared with simple baselines. The research does not collect data from customers, and it does not test the system with live users.

Time and Place of Research

The system development and the data analysis were carried out from October to November 2026 at the Informatics Study Program (S1), Universitas Bina Sarana Informatika (UBSI), Kaliabang Campus, Bekasi, Indonesia. The transactions analyzed were recorded from 1 January to 30 June 2023 (181 days) at the Lower Manhattan branch of a simulated coffee shop.

Research Target / Subject

The data come from the public Coffee Shop Sales dataset published by Maven Analytics under a public domain license. The publisher describes it as a simulated dataset for analytics practice, so it does not record real customers. The file has 149,116 transactions from three branches and 11 attributes: transaction ID, date, time, quantity, store ID, store location, product ID, unit price, product category, product type, and product detail. This study uses only the Lower Manhattan branch (store ID 5), which has 47,782 transactions, 80 products, and nine product categories. Figure 1 shows how the average number of transactions changes during the day. The two other branches are excluded so that the models describe one

The work is distributed under the Creative Commons Attribution-NonCommercial 4.0 International License

shop. No sampling is applied, and all 47,782 transactions of the branch are used. The dataset has no customer or basket identifier, so the study cannot analyze individual customers or items bought together.

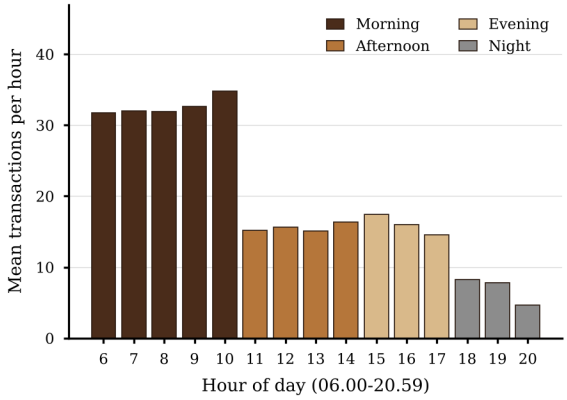


Figure 1. Mean Number of Transactions per Hour at the Lower Manhattan Branch, January-June 2023

Procedure

The research follows seven steps, summarized in Figure 2.

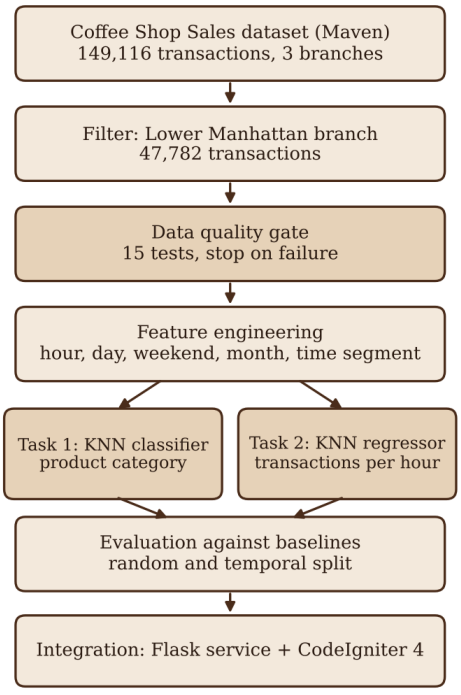


Figure 2. Research Procedure

(1) Data quality testing. Before any modeling, the data pass a quality gate of 15 tests: file integrity (SHA-256), schema, missing values, duplicates, date and time validity, numeric ranges and outliers, categorical and price consistency, transaction ID gaps, operating hours, class balance, monthly trend, dependence on time, target leakage, and row reconciliation. A test that fails stops the pipeline. Outliers that are legitimate, such as the high price of coffee beans, are kept and flagged.

(2) Feature engineering. The hour, the day of the week, a weekend indicator, and the month are taken from the transaction time and date. The hour is also encoded as a sine and cosine pair so that 20.59 and 06.00 are not treated as far apart on a daily cycle. The operating day is divided into four time segments: morning (06.00-10.59), afternoon (11.00-14.59), evening (15.00-17.59), and night (18.00-20.59). Requests outside these hours are handled by a fallback message. Product ID, unit price, product type, and product detail are not used as features of the category model, to prevent target leakage. Table 1 lists the features of each model.

Table 1. Features Used by Each Model

Feature	Description	Task 1	Task 2
Hour	Hour of the transaction (6-20)	Yes	Yes
Day of week	0 (Monday) to 6 (Sunday)	Yes	Yes
Weekend indicator	1 for Saturday and Sunday	Yes	Yes
Month	Month of the year (1-6)	No	Yes
Hour sine and cosine	Cyclic encoding of the hour	Yes	Yes

(3) Menu recommendation model (Task 1). A KNN classifier predicts the product category from the hour, day of the week, weekend indicator, and the sine and cosine of the hour. The best categories are then mapped to the most popular menu items of the current time segment.

The work is distributed under the Creative Commons Attribution-NonCommercial 4.0 International License

(4) Crowd prediction model (Task 2). Transactions are counted for every date and hour. A KNN regressor predicts the number of transactions per hour from the hour, day of the week, weekend indicator, month, and the sine and cosine of the hour. An hour is labeled quiet, moderate, or busy using the first and third quartiles of the training counts, which are 11 and 28 transactions per hour.

(5) System integration. The models are saved with their metadata and served by a Python Flask service on a local port, following the deployment approach of Suherlan et al. (2025). The CodeIgniter 4 application calls the service when a customer opens the menu of a table and shows the recommended items and the crowd label in a widget. If the service is unavailable or returns an invalid result, the application uses a rule-based fallback based on item popularity. Figure 3 shows the architecture. Each shown and selected recommendation is logged without customer identity. The database has tables for users, tables, products and their options, orders, order items, payments, notifications, product activity, settings, aggregated crowd and category data, and recommendation logs.

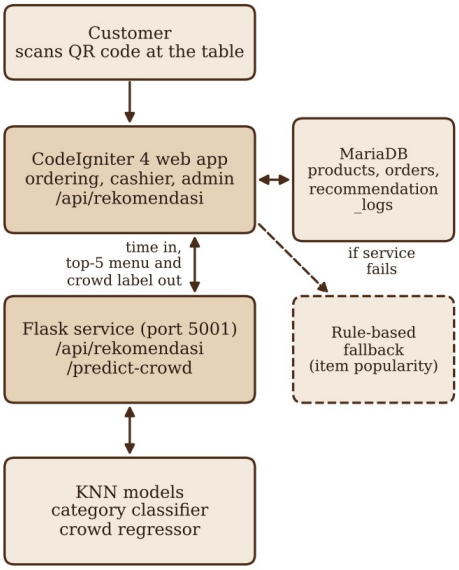


Figure 3. Architecture of the Recommendation System

(6) Evaluation against baselines. Each model is compared with simple baselines that need no learning, so that any gain from KNN can be measured.

(7) Interpretation. The results are read together with the tests of time dependence, and the limits of a simulated dataset are stated.

Data, Instruments, and Data Collection Techniques

The data are secondary quantitative data. The Excel file was downloaded once from the Maven Analytics data playground and was never modified, and its SHA-256 hash is stored so that every later run can be tied to the same file. The instruments are software tools: Python with pandas, NumPy, and scikit-learn for data processing and modeling, Matplotlib for figures, Flask for the model service, and CodeIgniter 4 on PHP 8.2 with MariaDB for the ordering application. The software versions used in training are saved in the model metadata, and random seeds are fixed so that the results can be repeated.

Data Analysis Technique

Features are standardized, because KNN depends on distances and the choice of scaling can

The work is distributed under the Creative Commons Attribution-NonCommercial 4.0 International License

change its results (Allorerung et al., 2024). The distance between two records x and y is the Euclidean distance, the square root of the sum of squared differences of all features. The number of neighbors K is chosen from 1 to 20 by five-fold cross-validation on the training data only (Hafid, 2023; Pratama & Mandita, 2024). The classifier is chosen by the highest mean accuracy, and the regressor by the lowest mean absolute error. The scaler and the encoder are fitted on training data only.

For Task 1, the data are split 80:20 with stratification and a fixed seed. A second test trains the model on January to May and tests it on June. The classifier is evaluated with accuracy, macro precision, macro recall, macro F1-score, weighted F1-score, a confusion matrix, and a top-3 hit rate, which counts a case as correct when the true category is among the three highest-ranked categories. The baselines are the majority category, the most frequent category of each time segment, and, for the top-3 hit rate, the three most frequent categories of the training data.

For Task 2, the hourly counts are evaluated with mean absolute error, root mean squared error, and the coefficient of determination (R^2). Two splits are used. The first is a random 80:20 split of the date-hour records. The second trains on January to May and tests on June, which is closer to real use because it predicts a month the model has not seen. The baselines are the mean count of each hour, and a seasonal naive baseline that uses the mean count of each hour in the last training month. A model without the month feature shows how much of the accuracy comes from the growth in volume over the months.

The dependence of the product category on the time segment is tested with the chi-square test of independence and measured with Cramér's V. The dependence of the hourly volume on the hour is tested with the Kruskal-Wallis test. These tests show whether time carries information about what customers order, about how many customers come, or both, and they explain why the two tasks are evaluated separately. Because the dataset is simulated, all conclusions are limited to the pipeline and to this dataset, in line with the cautions about synthetic data and offline evaluation raised by Figueira and Vaz (2022) and Hidasi and Czapp (2023).

REFERENCES

Allorerung, P. P., Erna, A., Bagussahrir, M., & Alam, S. (2024). Analisis performa normalisasi data untuk klasifikasi K-Nearest Neighbor pada dataset penyakit. *JISKA (Jurnal Informatika Sunan Kalijaga), 9*(3), 178-191. https://doi.org/10.14421/jiska.2024.9.3.178-191

Arisandi, D., Sutanto, K., & Perdana, N. J. (2024). Implementasi aplikasi pemesanan pada Life & Story Coffee untuk digitalisasi pencatatan pesanan. *Panrita Abdi - Jurnal Pengabdian pada Masyarakat, 8*(3), 686-693. https://doi.org/10.20956/pa.v8i3.26742

Bogina, V., Kuflik, T., Jannach, D., Bielikova, M., Kompan, M., & Trattner, C. (2023). Considering temporal aspects in recommender systems: A survey. *User Modeling and User-Adapted Interaction, 33*(1), 81-119. https://doi.org/10.1007/s11257-022-09335-w

Bondevik, J. N., Bennin, K. E., Babur, Ö., & Ersch, C. (2024). A systematic review on food recommender systems. *Expert Systems with Applications, 238*, 122166. https://doi.org/10.1016/j.eswa.2023.122166

Chow, Y. Y., Haw, S. C., Naveen, P., Anaam, E. A., & Mahdin, H. (2023). Food recommender system: A review on techniques, datasets and evaluation metrics. *Journal of System and Management Sciences, 13*(5). https://doi.org/10.33168/JSMS.2023.0510

Figueira, A., & Vaz, B. (2022). Survey on synthetic data generation, evaluation methods and GANs. *Mathematics, 10*(15), 2733. https://doi.org/10.3390/math10152733

Hafid, H. (2023). Penerapan K-fold cross validation untuk menganalisis kinerja algoritma K-Nearest Neighbor pada data kasus Covid-19 di Indonesia. *Journal of Mathematics Computations and Statistics, 6*(2), 161-168. https://doi.org/10.35580/jmathcos.v6i2.53043

The work is distributed under the Creative Commons Attribution-NonCommercial 4.0 International License

Hidasi, B., & Czapp, Á. T. (2023). Widespread flaws in offline evaluation of recommender systems. In *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys '23)*. Singapore: ACM.

Hssina, B., Grota, A., & Erritali, M. (2021). Recommendation system using the k-nearest neighbors and singular value decomposition algorithms. *International Journal of Electrical and Computer Engineering, 11*(6), 5541-5548. https://doi.org/10.11591/ijece.v11i6.pp5541-5548

Mellany, J. S., & Kusnawi, K. (2026). Hybrid food recommendation system using term frequency-inverse document frequency (TF-IDF), K-nearest neighbors (KNN), and tag-based similarity. *AVITEC, 8*(1), 151-161. https://doi.org/10.28989/avitec.v8i1.3683

Meng, X., Du, Y., Zhang, Y., & Han, X. (2022). A survey of context-aware recommender systems: From an evaluation perspective. IEEE Transactions on Knowledge and Data Engineering. Advance online publication. https://doi.org/10.1109/tkde.2022.3187434

Nafi, T. M., & Mujiyono, S. (2025). Aplikasi rekomendasi menu makanan harian menggunakan algoritma metode KNN. *Jurnal Algoritma, 22*(2), 381-388. https://doi.org/10.33364/algoritma/v.22-2.2491

Panwar, M., Sharma, A., Mahela, O. P., Khan, B., & Ali, A. (2024). An intelligent time aware food recommender system using support vector machine. *Indonesian Journal of Electrical Engineering and Computer Science, 34*(1), 620-629. https://doi.org/10.11591/ijeecs.v34.i1.pp620-629

Pratama, R. K., & Mandita, F. (2024). Klasifikasi penerimaan tenaga kerja tertutup menggunakan algoritma K-Nearest Neighbor (KNN). *Indexia, 6*(1), 42-61. https://doi.org/10.30587/indexia.v6i1.7224

Pratama, Y. C. A., & Dewi, C. (2025). Analysis of consumer purchasing patterns using the Apriori algorithm on sales transaction data from Anak Panah Kopi Salatiga. *International Journal Software Engineering and Computer Science, 5*(1), 1-10. https://doi.org/10.35870/ijsecs.v5i1.3275

Rizki, A. P., Auliasari, K., & Rudhistiar, D. (2024). Sistem rekomendasi penentuan paket menu menggunakan algoritma Apriori (studi kasus: IFF Ayam Uleg Cak Abit Cabang Bendungan Sutami). *JATI (Jurnal Mahasiswa Teknik Informatika), 8*(2), 1289-1297. https://doi.org/10.36040/jati.v8i2.9094

Salam, R. N., Dahliyanti, N., & Yel, M. B. (2026). Implementation of a website-based QR menu and self-ordering system at Warkop Abas. *International Journal Software Engineering and Computer Science, 6*(3), 1039-1045. https://doi.org/10.35870/ijsecs.v6i3.8325

Sari, N., Hananto, A., Priatna, B., & Hilabi, S. S. (2025). Implementasi algoritma K-NN untuk klasifikasi penjualan menu terlaris di RM Ratu Chaniago. *JIKA (Jurnal Informatika), 9*(3), 339-346. https://doi.org/10.31000/jika.v9i3.14445

Suherlan, E., Arti, S., Nabilah, S., & Hazimah, Z. (2025). Penerapan Flask framework untuk deployment model machine learning dalam mendukung analisis adaptasi mahasiswa pada pembelajaran daring. *PINTER: Jurnal Pendidikan Teknik Informatika dan Komputer, 9*(1), 110-117. https://doi.org/10.21009/pinter.9.1.15

Taslim, T., Gumanti, A. E., Handayani, S., & Toresa, D. (2022). Penerapan algoritma K-Nearest Neighbor untuk klasifikasi topik skripsi mahasiswa di Fakultas Ilmu Komputer. *JITACS: Journal of Information Technology and Computer Science, 1*(2), 45-50. https://doi.org/10.31849/jitacs.v1i2.10037

The work is distributed under the Creative Commons Attribution-NonCommercial 4.0 International License